

IPC144 – Introduction to C Programming Practice Exercises

Week 9 – Characters and Strings

8.5 (Character Testing) Write a program that inputs a character from the keyboard and tests the character with each of the functions in the character-handling library. The program should print the value returned by each function.

8.6 (Displaying Strings in Uppercase and Lowercase) Write a program that inputs a line of text into char array s[100]. Output the line in uppercase letters and in lowercase letters.

8.7 (Converting Strings to Integers for Calculations) Write a program that inputs four strings that represent integers, converts the strings to integers, sums the values and prints the total of the four values.

8.8 (Converting Strings to Floating Point for Calculations) Write a program that inputs four strings that represent floating-point values, converts the strings to double values, sums the values and prints the total of the four values.

8.9 (Comparing Strings) Write a program that uses function strcmp to compare two strings input by the user. The program should state whether the first string is less than, equal to or greater than the second string.

8.10 (Comparing Portions of Strings) Write a program that uses function strncmp to compare two strings input by the user. The program should input the number of characters to be compared, then display whether the first string is less than, equal to or greater than the second string.

8.11 (Random Sentences) Write a program that uses random number generation to create sentences. The program should use four arrays of pointers to char called article, noun, verb and preposition. The program should create a sentence by selecting a word at random from each array in the following order: article, noun, verb, preposition, article and noun. As each word is picked, it should be concatenated to the previous words in an array large enough to hold the entire sentence. The words should be separated by spaces. When the final sentence is output, it should start with a capital letter and end with a period. The program should generate 20 such sentences. The arrays should be filled as follows: The article array should contain the articles "the", "a", "one", "some" and "any"; the noun array should contain the nouns "boy", "girl", "dog", "town" and "car"; the verb array should contain the verbs "drove", "jumped", "ran", "walked" and "skipped"; the preposition array should contain the prepositions "to", "from", "over", "under" and "on".

After the preceding program is written and working, modify it to produce a short story consisting of several of these sentences. (How about the possibility of a random term paper writer?)

8.12 (Limericks) A limerick is a humorous five-line verse in which the first and second lines rhyme with the fifth, and the third line rhymes with the fourth. Using techniques similar to those developed in Exercise 8.11, write a program that produces random limericks. Polishing this program to produce good limericks is a challenging problem, but the result

will be worth the effort!

8.13 (Pig Latin) Write a program that encodes English-language phrases into pig Latin. Pig Latin is a form of coded language often used for amusement. Many variations exist in the methods used to form pig-Latin phrases. For simplicity, use the following algorithm: To form a pig-Latin phrase from an English-language phrase, tokenize the phrase into words with function `strtok`. To translate each English word into a pig-Latin word, place the first letter of the English word at the end of the English word and add the letters “ay.” Thus the word “jump” becomes “umpjay,” the word “the” becomes “hetay” and the word “computer” becomes “omputercay.” Blanks between words remain as blanks. Assume the following: The English phrase consists of words separated by blanks, there are no punctuation marks, and all words have two or more letters. Function `printLatinWord` should display each word. [Hint: Each time a token is found in a call to `strtok`, pass the token pointer to function `printLatinWord`, and print the pig-Latin word.]
Note: We’ve provided simplified rules for converting words to Pig Latin here. For more detailed rules and variations, visit en.wikipedia.org/wiki/Pig_latin.]

8.14 (Tokenizing Telephone Numbers) Write a program that inputs a telephone number as a string in the form (555) 555-5555. The program should use function `strtok` to extract the area code as a token, the first three digits of the phone number as a token and the last four digits of the phone number as a token. The seven digits of the phone number should be concatenated into one string. The program should convert the area-code string to `int` and convert the phone-number string to `long`. Both the area code and the phone number should be printed.

8.15 (Displaying a Sentence with Its Words Reversed) Write a program that inputs a line of text, tokenizes the line with function `strtok` and outputs the tokens in reverse order.

8.16 (Searching for Substrings) Write a program that inputs a line of text and a search string from the keyboard. Using function `strstr`, locate the first occurrence of the search string in the line of text, and assign the location to variable `searchPtr` of type `char *`. If the search string is found, print the remainder of the line of text beginning with the search string. Then, use `strstr` again to locate the next occurrence of the search string in the line of text. If a second occurrence is found, print the remainder of the line of text beginning with the second occurrence. [Hint: The second call to `strstr` should contain `searchPtr + 1` as its first argument.]

8.17 (Counting the Occurrences of a Substring) Write a program based on the program of Exercise 8.16 that inputs several lines of text and a search string and uses function `strstr` to determine the total occurrences of the string in the lines of text. Print the result.

8.18 (Counting the Occurrences of a Character) Write a program that inputs several lines of text and a search character and uses function `strchr` to determine the total occurrences of the character in the lines of text.

8.19 (Counting the Letters of the Alphabet in a String) Write a program based on the program of Exercise 8.18 that inputs several lines of text and uses function `strchr` to determine the total occurrences of each letter of the alphabet in the lines of text. Uppercase and lowercase letters should be counted together. Store the totals for each letter in an array and print the values in tabular format after the totals have been determined.

8.20 (Counting the Number of Words in a String) Write a program that inputs several lines of text and uses `strtok` to count the total number of words. Assume that the words are

separated by either spaces or newline characters.

8.21 (Alphabetizing a List of Strings) Use the string-comparison functions discussed in Section 8.6 and the techniques for sorting arrays developed in Chapter 6 to write a program that alphabetizes a list of strings. Use the names of 10 or 15 towns in your area as data for your program.

8.22 The chart in Appendix B shows the numeric code representations for the characters in the ASCII character set. Study this chart and then state whether each of the following is true or false.

- a) The letter "A" comes before the letter "B."
- b) The digit "9" comes before the digit "0."
- c) The commonly used symbols for addition, subtraction, multiplication and division all come before any of the digits.
- d) The digits come before the letters.
- e) If a sort program sorts strings into ascending sequence, then the program will place the symbol for a right parenthesis before the symbol for a left parenthesis.

8.23 (Strings Starting with "b") Write a program that reads a series of strings and prints only those strings beginning with the letter "b."

8.24 (Strings Ending with "ed") Write a program that reads a series of strings and prints only those strings that end with the letters "ed."

8.25 (Printing Letters for Various ASCII Codes) Write a program that inputs an ASCII code and prints the corresponding character. Modify this program so that it generates all possible three-digit codes in the range 000 to 255 and attempts to print the corresponding characters. What happens when this program is run?

8.26 (Write Your Own Character-Handling Functions) Using the ASCII character chart in Appendix B as a guide, write your own versions of the character-handling functions in Fig. 8.1.

8.27 (Write Your String Conversion Functions) Write your own versions of the functions in Fig. 8.5 for converting strings to numbers.

8.28 (Write Your Own String Copy and Concatenation Functions) Write two versions of each of the string-copy and string-concatenation functions in Fig. 8.17. The first version should use array subscripting, and the second version should use pointers and pointer arithmetic.

8.29 (Write Your Own Character and String I/O Functions) Write your own versions of the functions getchar, putchar and puts described in Fig. 8.12.

8.30 (Write Your Own String Comparison Functions) Write two versions of each string-comparison function in Fig. 8.20. The first version should use array subscripting, and the second version should use pointers and pointer arithmetic.

8.31 (Write Your Own String Searching Functions) Write your own versions of the functions in Fig. 8.22 for searching strings.

8.32 (Write Your Own Memory-Handling Functions) Write your own versions of the functions in Fig. 8.30 for manipulating blocks of memory.

8.33 (Write Your Own String Length Function) Write two versions of function `strlen` in Fig. 8.36. The first version should use array subscripting, and the second version should use pointers and pointer arithmetic.